

HW#4 RTOS Analysis

鄭維, 10801069

Abstract—This report analyzes the context switching and synchronization mechanisms of FreeRTOS on the Aquila RISC-V microprocessor. Results show that increasing the time quantum reduces context switching overhead and interrupt frequency, while latency per interrupt remains unaffected. Although a 20 ms time quantum minimizes the impact of context switching during synchronization profiling, the measurements of synchronization overhead are still imprecise. Future work could improve accuracy by implementing a more effective profiling mechanism, ensuring precise analysis of synchronization performance.

I. INTRODUCTION

In this report, I examine the context switching and synchronization mechanisms of FreeRTOS on the Aquila RISC-V microprocessor. The context switching analysis focuses on task creation, scheduler initialization, and interrupt handling. Synchronization is studied through the use of mutex operations to ensure safe access to shared resources. Experiments were conducted by measuring overheads with different time quantum values to evaluate their impact on performance.

II. CONTEXT SWITCHING BEHAVIOR ANALYSIS

This discussion explores the context switching algorithm in FreeRTOS by examining the source code of a demo program, `rtos_run`, on a RISC-V architecture. The context switching is illustrated through the task creation and scheduler initialization processes in the main function. Fig. 1. shows the flow chart of context switching

A. Task Creation and Initialization

In the FreeRTOS, tasks are created using the `xTaskCreate()` function. The `Task1_Handler` and `Task2_Handler` functions are assigned to tasks with priority levels of 3, and the scheduler is initialized with `vTaskStartScheduler()`. Internally, task creation initializes task control blocks (TCBs), which are critical data structures used to manage task states and events. Each TCB contains members such as `xStateListItem` for task state management (e.g., ready, blocked, suspended) and `xEventListItem` for event handling.

The `prvInitialiseNewTask()` function configures the TCB's initial state and sets its event priority. Subsequently, `prvAddNewTaskToReadyList()` adds the task to the appropriate ready list, organized by priority levels within the `pxReadyTasksLists` array. If it is the first task being added, the system initializes various task lists, including `pxDelayedTaskList`, `pxOverflowDelayedTaskList`, and `xSuspendedTaskList`. The highest-priority task is designated as `pxCurrentTCB` if the scheduler is not running.

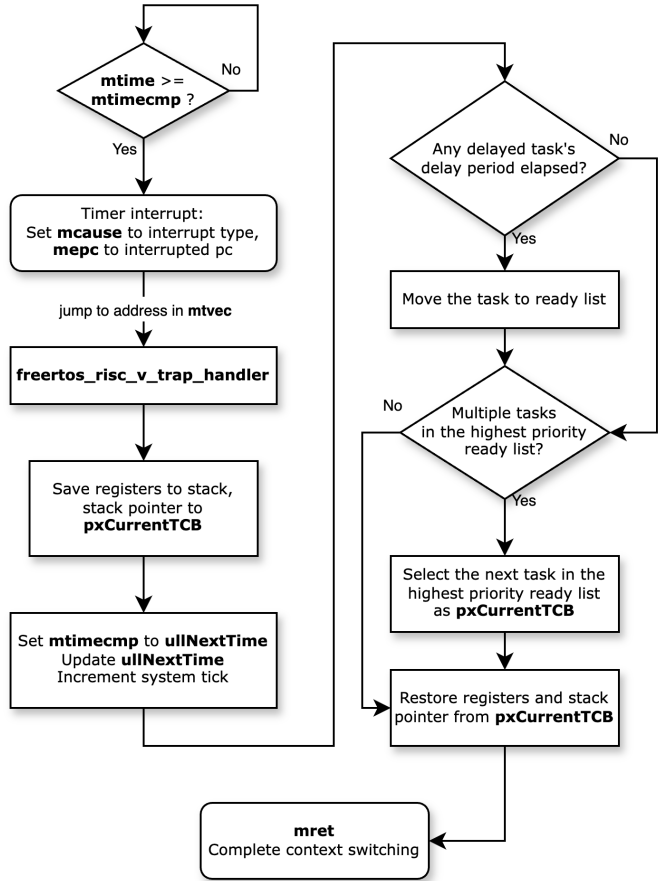


Fig. 1. Flow chart of Interrupt Handling and Context Switching

B. Scheduler Initialization

The `vTaskStartScheduler()` function activates the FreeRTOS scheduler. This process involves creating the idle task, which continuously checks for tasks awaiting termination. Although the `prvTimerTask` is initialized, it remains in a suspended state if no timer operations are required. The `xPortStartScheduler()` function, which directly interacts with hardware, sets up the timer interrupt in CLINT. Specifically, the `mtimecmp` register in CLINT is configured to trigger periodic timer interrupts based on the system's tick rate, defined by `configTICK_RATE_HZ`.

Following the timer setup, the `xPortStartFirstTask()` function, implemented in 'portASM.S'(RISC-V assembly), begins the scheduler by loading the `freertos_risc_v_trap_handler()` address into the `mtvec` register. It restores the registers of the first task from the TCB and starts execution by returning to the task's program counter (PC) address stored in `ra`.

C. Interrupt Handling and Context Switching

The `freertos_risc_v_trap_handler()`, also implemented in 'portASM.S', is a pivotal component in FreeRTOS, responsible for managing hardware interrupts and initiating context switches. When a timer interrupt occurs, triggered by the condition (`mtime >= mtimecmp`) in CLINT, the handler saves the current task's context, including registers and stack pointer, into its TCB.

The handler distinguishes between synchronous and asynchronous interrupts using the `mcause` register. For timer interrupts, it updates `mtimecmp` to schedule the next tick and invokes `xTaskIncrementTick` to increment the system tick count. This function evaluates tasks in the delayed list and moves them to the ready list if their delay period has elapsed. If multiple tasks of the same priority exist in the ready list, time slicing is applied by explicitly setting `xSwitchRequired`, which ensures fair execution and enforces proper sharing of the CPU time among tasks of the same priority.

If a context switching is required, indicated by `xSwitchRequired`, the `vTaskSwitchContext()` function selects the highest-priority task from the `pxReadyTasksLists`. It updates `pxCurrentTCB` to point to the new task and restores its context, including the stack pointer, registers, and program counter. The `mret` instruction then resumes execution of the selected task, completing the context switching process.

D. Priority Management

FreeRTOS prioritizes tasks based on their assigned priority levels. In this implementation, Task1 and Task2 share a priority of 3, while the idle task is assigned a priority of 0. Timer-related tasks are given a higher priority of 6 (though suspended). The scheduler dynamically adjusts execution based on priority levels, ensuring that higher-priority tasks preempt lower-priority tasks and that tasks of equal priority share CPU time through round-robin scheduling.

III. SYNCHRONIZATION BEHAVIOR ANALYSIS

The following discussion explores the synchronization mechanism in FreeRTOS by examining the shared access to `shared_counter` between `Task1_Handler` and `Task2_Handler`. Fig. 2. shows the flow chart of mutex take.

A. Mutex Data Structure

In FreeRTOS, the `xQueueDefinition` structure serves as the foundation for implementing both queues and synchronization mechanisms such as semaphores and mutexes. This structure is referenced using aliases like `Queue_t` and `QueueHandle_t`. The `union u` member differentiates the usage context by providing either `QueuePointers_t xQueue` for queue operations or `SemaphoreData_t xSemaphore` for synchronization. Within the semaphore context, the `xSemaphore` structure includes `xMutexHolder`, which identifies the task currently holding the mutex, and `uxRecursiveCallCount`, which tracks the number of recursive mutex acquisitions.

Additionally, `xQueueDefinition` features `xTaskWaitingToReceive`, an event list of tasks waiting to acquire the semaphore, and `uxMessagesWaiting`, a counter

that represents the availability of the mutex (set to 1 when the mutex is unowned and 0 when owned).

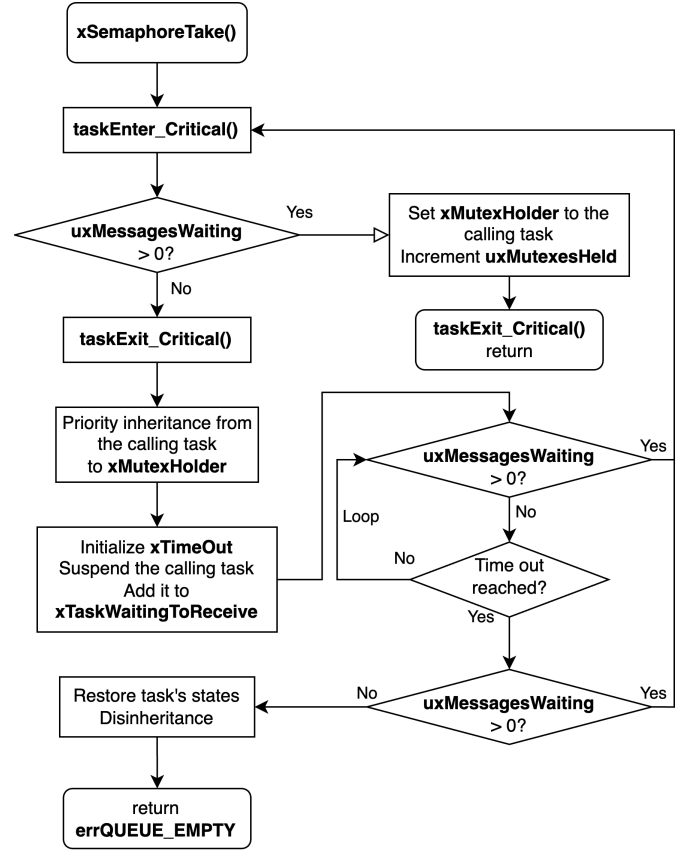


Fig. 2. Flow chart of Mutex Take

B. Creating a Mutex

The creation of a mutex in FreeRTOS is handled by the `xQueueCreateMutex()` function. This function allocates memory for the mutex using `pvPortMalloc()` and initializes the structure using `prvInitialiseQueue()`. During initialization, the mutex is configured with default attributes: `xMutexHolder` is set to `NULL`, indicating that no task owns the mutex, and `uxRecursiveCallCount` is set to zero to allow recursive usage.

The final step in mutex creation involves calling `xQueueGenericSend()` within the `prvInitialiseMutex()` function. This call initializes the mutex by simulating its release, setting the `uxMessagesWaiting` counter to 1. This action ensures that the mutex is ready to be acquired by tasks once created. By leveraging the same function used for releasing a mutex, FreeRTOS ensures consistency in mutex initialization and release behaviors.

C. Mutex Take

When a task requires exclusive access to a resource, it calls `xSemaphoreTake()` to take the mutex. This function begins by entering a critical section with `taskENTER_CRITICAL()`, preventing context switching during the operation. If the mutex is available (`uxMessagesWaiting > 0`), the function updates the `xMutexHolder` to reference the calling task, decrements `uxMessagesWaiting`, and increments the task's `uxMutexesHeld` count. It then exits the critical section and

returns, allowing the task to proceed with exclusive access to the resource.

If the mutex is unavailable (**uxMessagesWaiting** == 0), the function initializes a timeout structure **xTimeOut** and suspends the calling task, adding its task control block to the **xTaskWaitingToReceive** event list of the mutex. To prevent priority inversion, the priority of the task currently holding the mutex **xMutexHolder** is temporarily raised to at least match the priority of the waiting task. This adjustment ensures that the owning task will not be preempted by lower-priority tasks, allowing it to release the mutex promptly. If the mutex becomes available before the timeout expires, the waiting task is resumed and allowed to acquire the mutex. If the timeout is reached and the mutex remained occupied, the function returns an error code **errQUEUE_EMPTY** and restores the task's state, including reverting any priority inheritance adjustments made during the operation.

D. Mutex Give

Releasing a mutex is handled by **xSemaphoreGive()**, which ensures that the resource is made available to other tasks. Within a critical section, the function updates the **xMutexHolder** to **NULL**, resets the owning task's inherited priority (if applicable), and increments the **uxMessagesWaiting** counter to indicate that the mutex is now available. If tasks are waiting for the mutex, the highest-priority task in the **xTaskWaitingToReceive** list is removed and made ready to execute. The function concludes by exiting the critical section and returning, ensuring a smooth transition of resource ownership.

IV. EXPERIMENTAL SETUP

The **FreeRTOS** kernel and the demo program **run_rtos** were compiled using the **GCC 13.2.0** compiler. To measure the overhead of context switching and synchronization, a hardware profiler was implemented within **Aquila**. Each experiment was executed five times, with one initial warm-up run excluded from the calculations. The results are the average values of the remaining five executions.

The total execution cycles were measured using defined start and end points. The starting point was marked at the entry of the main function. Since **FreeRTOS**, as an operating system, does not terminate execution naturally, the endpoint was set at the entry of the **vTaskDelete** function in the **Task1_Handler**. This was chosen because **Task1** terminates after **Task2**, providing a reasonable cycle measurements.

A. Context Switching Analysis

Each **tmr_irq_0** signal generated by the **clint.v** module indicated a timer interrupt. The context switching overhead was determined by counting the clock cycles from the occurrence of a timer interrupt until the next **mret** instruction. To evaluate the impact of varying time quantum sizes on overhead, three time quantum durations—5 ms, 10 ms, and 20 ms—were profiled and compared.

B. Synchronization Analysis

The **xSemaphoreTake()** function was profiled to measure the overhead for mutex acquisition, while the

xSemaphoreGive() function was profiled to measure the overhead for mutex release. The total synchronization overhead was calculated as the sum of the cycles spent on both mutex take and give operations.

To minimize interference from context switching, a time quantum of 20 ms was selected, as it exhibited the least context switching overhead during the experiments. However, it is important to note that the profiling mechanism might inadvertently include some cycles related to context switching, which could affect the accuracy of the synchronization overhead measurements.

V. RESULTS AND DISCUSSION

TABLE I. CONTEXT SWITCHING ANALYSIS

Time Quantum	5 ms	10 ms	20 ms
Latency	1047753.0	462058.6	258395.6
Total Share (%)	5.18	2.29	1.28
Timer IRQ count	969.6	484	242.2
Latency per IRQ	1080.60	954.67	1066.87

The results of the context-switching analysis are presented in Table 1. The total share of context-switching overhead decreases as the time quantum increases. This trend is accompanied by a reduction in the Timer IRQ count. This behavior can be attributed to the fact that increasing the time quantum reduces the frequency of timer interrupts, thereby decreasing the number of context-switching events.

Interestingly, the latency per Timer IRQ doesn't show significant variation across different time quantum settings. This consistency implies that the workload for each timer interrupt handling remains similar, regardless of whether it results in context switching or not.

TABLE II. SYNCHRONIZATION ANALYSIS

Synchronization	Mutex Take	Mutex Give	Mutex
Latency	1891648.6	1950403.0	3842051.6
Total Share (%)	9.35	9.64	19.00

The synchronization analysis results are shown in Table II.

VI. CONCLUSION

This report demonstrates that increasing the time quantum reduces context switching overhead and timer interrupt frequency without affecting the latency per interrupt. However, despite selecting a 20 ms time quantum to minimize the impact of context-switching overhead, the synchronization overhead measurements remain imprecise. Future improvements could focus on developing an appropriate profiling mechanism to accurately analyze synchronization overhead.